



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

پروژه درس شبکه‌های کامپیوتری
نیمسال اول سال تحصیلی ۱۴۰۴-۱۴۰۵

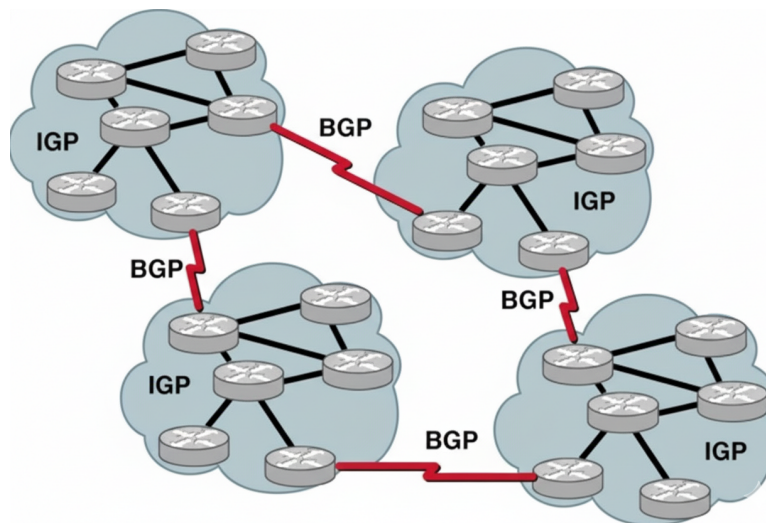
استاد: دکتر باطنی

دستیاران آموزشی:

آریا جلو دار
احمد عمرانی
امیر حسین جعفری
امیر حسین معلمی
ریحانه شیرانی
سپهر رجبی
سنا باقری
محمد رهی
مهدی رهبر
مهرزاد انصاری پور
میعاد کیمیاگری

پاییز ۱۴۰۴

فاز دوم: پیاده‌سازی و شبیه‌سازی پروتکل‌های مسیریابی (لایه شبکه)



۱-۱ شرح کلی فاز دوم

در فاز دوم، تمرکز از لایه انتقال و سوکت‌های واقعی به لایه شبکه و Routing جابه‌جا می‌شود، اما همه چیز همچنان در حد یک شبیه‌ساز نرم‌افزاری داخل برنامه شما انجام می‌شود. یعنی قرار نیست با IP Stack سیستم عامل، سوکت‌های خام یا تنظیم روتر واقعی سروکار داشته باشید، تمام روترها، لینک‌ها و پیام‌ها صرفاً آبجکت‌ها و ساختارهای داده در کد شما هستند.

در این فاز یک شبیه‌ساز شبکه طراحی می‌کنید که در آن چندین Router مجازی از طریق لینک‌های وزن‌دار به هم متصل شده‌اند و با تبادل پیام‌های کنترلی، جدول‌های مسیریابی (Routing Table) خود را می‌سازند. تمرکز فاز روی الگوریتم مسیریابی حالت-لینک (Link-State) است: هر روتر با ارسال و دریافت پیام‌های LSA (Link State Advertisement) تصویر مشترکی از توپولوژی شبکه به دست می‌آورد و سپس با اجرای الگوریتم Dijkstra روی این تصویر، کوتاه‌ترین مسیرها را به تمام روترهای دیگر حساب می‌کند.

خروجی این فاز یک برنامه است که وقتی روی یک توپولوژی کوچک اجرا شود، بتوانید در آن:

- چند روتر مجازی و لینک‌های وزن‌دار بین آن‌ها را تعریف کنید.
- انتشار پیام‌های LSA و ساخت Link-State Database در هر روتر را مشاهده کنید.
- جدول‌های مسیریابی حاصل از اجرای Dijkstra را ببینید.
- و در ساده‌ترین حالت، عبور یک بسته داده آزمایشی از روتر مبدأ به مقصد را بر اساس همین جدول‌ها شبیه‌سازی کنید.

برای این فاز کافی است بتوانید **حداقل یک سناریوی ساده** را از ابتدا تا انتها اجرا کنید. نیازی نیست چندین توپولوژی یا سناریوهای متنوع آماده کنید. اما شبیه‌ساز شما باید به‌گونه‌ای طراحی شود که اگر بعداً توپولوژی را عوض کردید، هنوز بتواند مراحل ساخت گراف انتشار LSA اجرای Dijkstra را انجام دهد.

تاکید می‌کنیم که در این فاز **هیچ الزامی** برای اجرای برنامه روی چند ماشین، استفاده از سوکت‌های شبکه یا تست روی اینترنت وجود ندارد، اگر همه روترها به‌صورت آبجکت‌های درون یک پروسه واحد پیاده‌سازی شوند کاملاً کافی است.

۲-۱ اهداف پروژه

هدف اصلی این فاز این است که مفاهیم مسیریابی لایه شبکه را از حالت رسم شکل و فرمول، به یک سیستم اجرایی قابل مشاهده تبدیل کنید. در پایان این فاز انتظار می‌رود:

- مفهوم Routing و نقش Routerها در یک شبکه چندگامی را به‌صورت عملی درک کنید.
- بتوانید توپولوژی شبکه را به‌صورت یک Graph وزن‌دار مدل کرده و تفاوت گره، لینک و هزینه مسیر را توضیح دهید.
- الگوریتم Link-State را در عمل پیاده‌سازی کنید: از تولید LSA تا Flooding و ساخت Link-State Database.
- الگوریتم Dijkstra را برای محاسبه Shortest Path Tree از دید هر روتر پیاده‌سازی و دیباگ کنید.
- درک کیفی از Convergence (همگرایی)، هزینه پیام‌ها و رفتار پروتکل هنگام تغییر توپولوژی به‌دست آورید.
- و در صورت علاقه، با ایده‌های اولیه Distance-Vector، OSPF و BGP در قالب بخش‌های امتیازی آشنا شوید.

۳-۱ الزامات حداقلی و بخش‌های اختیاری

در این بخش، حداقل چیزهایی را که پروژه شما **حتما** باید داشته باشد، و همچنین چند مسیر توسعه اختیاری را مشخص می‌کنیم.

۱-۳-۱ بخش اجباری

۱. پیاده‌سازی یک شبیه‌ساز شبکه شامل چندین روتر مجازی.

این شبیه‌ساز **کاملا نرم‌افزاری** است و در یک برنامه واحد اجرا می‌شود. روترها آبجکت‌هایی در کد شما هستند و لینک‌ها صرفا یال‌های یک گراف وزن‌دار. نیازی به استفاده از سوکت‌های واقعی، تنظیم IP یا دسترسی سطح سیستم‌عامل نیست. پیشنهاد می‌شود:

- یک توپولوژی کوچک ولی غیر بدیهی (مثلا ۵ تا ۸ روتر، با دست‌کم یک حلقه) تعریف کنید.
- توپولوژی را می‌توانید مستقیما در کد یا از یک فایل ساده (اختیاری) بخوانید.
- مدل زمان شبیه‌سازی می‌تواند یک حلقه ساده باشد که در آن پیام‌ها را یکی‌یکی پردازش می‌کنید و بعد از اتمام تبادل LSA سراغ اجرای Dijkstra می‌روید.

۲. هر روتر باید یک جدول مسیریابی (Routing Table) داشته باشد.

جدول مسیریابی می‌تواند یک ساختار ساده مانند Dictionary یا آرایه‌ای از رکوردها باشد که حداقل فیلدهای زیر را نگه می‌دارد:

- شناسه مقصد.

• Next Hop (روتر بعدی در مسیر به سمت آن مقصد).

• هزینه کل مسیر (جمع وزن لینک‌ها).

لازم نیست پیاده‌سازی جدول بسیار پیشرفته باشد، کافی است بتوانید آن را برای چند روتر چاپ کنید و از روی آن مسیر یک بسته آزمایشی را تعیین کنید. اگر خواستید می‌توانید فیلدهای دیگری مثل تعداد Hop یا توضیح مسیر را هم به صورت اختیاری اضافه کنید.

۳. پیاده‌سازی الگوریتم مسیریابی حالت-لینک (Link-State): استفاده از الگوریتم Dijkstra برای محاسبه کوتاه‌ترین مسیر به تمامی گره‌های مقصد.

مثال: هر روتر با استفاده از پروتکل‌های شبیه‌سازی شده (مثلاً با ارسال پیام‌های LSA Link State Advertisement - به همه روترها)، یک دید کامل از توپولوژی شبکه بدست می‌آورد.

برای رسیدن به این بند، می‌توانید این مراحل را در کد پیاده‌سازی می‌کنید (جزئیات به انتخاب شماست):

- تعریف قالب ساده برای پیام LSA (شامل شناسه مبدأ، فهرست همسایه‌ها و هزینه لینک‌ها، و یک Seq Number برای تشخیص جدیدتر بودن پیام‌ها).
- Flooding پیام‌های LSA در شبکه شبیه‌سازی شده تا همه روترها اطلاعات یکسانی از توپولوژی داشته باشند (هر روتر LSA جدید را فقط یک بار پردازش و دوباره ارسال می‌کند).
- ساخت Link-State Database در هر روتر و اجرای الگوریتم Dijkstra روی این گراف از دید همان روتر به عنوان مبدأ.
- استخراج Next Hop و هزینه مسیر از خروجی Dijkstra و پر کردن Routing Table.

۱-۳-۲ قسمت امتیازی (اختیاری)

برای این‌که تصویر کلی داشته باشید، برخی امکاناتی که می‌توانند بعداً به صورت امتیازی اضافه شوند (و در ادامه سند توضیح بیشتری خواهند داشت) عبارت‌اند از:

- پیاده‌سازی الگوریتم Distance-Vector ساده شبیه RIP.
- شبیه‌سازی مفاهیم ساده‌ای از OSPF مانند Area.
- شبیه‌سازی بسیار ساده‌ای از BGP بین چند AS.
- بصری‌سازی توپولوژی و مسیرها.
- اضافه کردن رخدادهای دینامیک مثل Link Failure (اختیاری).

۴-۱ تعاریف مورد نیاز

۱-۴-۱ مفاهیم پایه شبیه‌سازی شبکه و توپولوژی

- **Router**: گره‌ای در شبکه که بسته‌ها را بر اساس Routing Table خود از یک رابط به رابط دیگر هدایت می‌کند. در این پروژه، روترها کاملاً مجازی و درون شبیه‌ساز شما هستند.
- **Link**: ارتباط منطقی بین دو روتر که می‌تواند وزن یا هزینه داشته باشد.
- **Topology**: ساختار گرافی شبکه، این‌که کدام روترها به هم وصل هستند و هزینه هر لینک چقدر است.
- **Graph**: مدل ریاضی شبکه، شامل مجموعه‌ای از Node ها (روترها) و Edge ها (لینک‌ها) که هر Edge می‌تواند وزن داشته باشد.
- **Cost/Metric**: عددی که کیفیت یک لینک یا مسیر را نشان می‌دهد. الگوریتم‌های مسیریابی تلاش می‌کنند مسیری با کمترین Cost را پیدا کنند.
- **Message/Packet**: واحد اطلاعاتی که در شبیه‌ساز بین روترها ردوبدل می‌شود، می‌تواند پیام کنترلی (LSA) یا یک بسته داده آزمایشی باشد.
- **Tick/Simulation Step**: واحد زمانی انتزاعی در شبیه‌ساز. در هر Tick می‌توانید پیام‌ها را یک‌به‌یک پردازش کنید، در ساده‌ترین حالت، می‌توانید بدون مدل زمانی پیچیده، فقط چند مرحله منطقی پشت‌سرهم اجرا کنید (انتشار LSA، سپس Dijkstra، سپس ارسال بسته آزمایشی).

۲-۴-۱ مفاهیم Link-State و Dijkstra

- **Link-State Routing**: خانواده‌ای از الگوریتم‌های مسیریابی که در آن هر روتر وضعیت لینک‌هایش را به دیگران اعلام می‌کند تا همه یک دید تقریباً مشترک از توپولوژی داشته باشند.
- **LSA (Link State Advertisement)**: پیامی که یک روتر در آن فهرست همسایه‌های خود و هزینه لینک‌ها را منتشر می‌کند.
- **Flooding**: مکانیزمی برای انتشار پیام‌ها در سراسر شبکه، هر روتر پس از دریافت یک LSA جدید (و معتبر)، آن را برای همسایه‌های خود می‌فرستد.

- **Link-State Database**: مجموعه LSAهای معتبر که در یک روتر ذخیره شده و گراف کامل شبکه از دید آن روتر را تشکیل می‌دهد.
- **Dijkstra**: الگوریتم کلاسیک برای پیدا کردن کوتاه‌ترین مسیر از یک مبدأ به همه مقصدها در گراف وزن‌دار، خروجی آن معمولاً شامل کمینه هزینه و والد هر گره در درخت است.
- **Shortest Path Tree (SPT)**: درختی ریشه‌دار (ریشه: روتر مبدأ) که برای هر گره کوتاه‌ترین مسیر از مبدأ تا آن گره را نشان می‌دهد. برای ساخت Routing Table، از همین درخت استفاده می‌کنید تا برای هر مقصد، Next Hop را پیدا کنید.

۳-۴-۱ مفاهیم Distance-Vector (برای بخش امتیازی)

- **Distance-Vector Routing**: رویکردی که در آن هر روتر فقط فاصله خود تا مقصدهای مختلف را می‌داند و این بردار فاصله را به همسایه‌هایش اعلام می‌کند.
- **Distance Vector**: آرایه‌ای از هزینه‌ها که در آن برای هر مقصد، هزینه رسیدن از روتر فعلی نوشته شده است.
- **RIP**: پروتکل ساده‌ای از خانواده Distance-Vector که از تعداد Hop به عنوان Metric استفاده می‌کند.
- **Count-to-Infinity**: مشکلی کلاسیک در Distance-Vector که در آن به دلیل اطلاعات ناکامل، هزینه یک مسیر نامعتبر ممکن است آرام‌آرام تا مقدار بزرگی بالا برود.

۴-۴-۱ مفاهیم OSPF و BGP (برای مطالعه بیشتر و امتیازی)

- **OSPF**: یک پروتکل Link-State واقعی و پرکاربرد در شبکه‌های درون‌دامنه‌ای که شبکه را به چند Area تقسیم می‌کند و درون هر Area تقریباً همان منطق نقشه کامل به همراه Dijkstra را دارد.
- **Area**: بخش‌بندی منطقی شبکه در OSPF برای مدیریت مقیاس، داخل هر Area اطلاعات لینک‌ها کامل است و فقط خلاصه اطلاعات بین نواحی ردوبدل می‌شود.
- **BGP**: پروتکل اصلی مسیریابی بین‌دامنه‌ای (Inter-domain) در اینترنت که مسیرها را بین ASهای مختلف مبادله می‌کند.
- **AS (Autonomous System)**: مجموعه‌ای از روترها یا شبکه‌ها که تحت مدیریت یک سازمان واحد قرار دارند.

• **AS-Path و Policy:** اطلاعاتی که BGP برای جلوگیری از حلقه و اعمال سیاست‌های مسیریابی (مثلا ترجیح مسیرهای خاص، یا اجتناب از عبور از یک AS مشخص) استفاده می‌کند.

۱-۵ مرور مفهومی: از Routing Table تا Link-State

۱-۵-۱ چرا به پروتکل‌های مسیریابی نیاز داریم؟

در یک شبکه کوچک دو یا سه گره‌ای، شاید بتوانید با چشم تصمیم بگیرید هر بسته از کدام مسیر برود، مثلا اگر فقط سه روتر A، B و C داشته باشید و A فقط به B متصل باشد، واضح است که بسته ارسالی از A به C باید از B عبور کند.
از طرفی، شبکه پویاست:

- لینک‌ها خراب می‌شوند یا کیفیت‌شان عوض می‌شود.
 - روترهای جدید اضافه می‌شوند یا برخی از کار می‌افتند.
 - و ما نمی‌خواهیم بعد از هر تغییر، جدول‌های مسیریابی را دستی به‌روز کنیم.
- Routing Protocol‌ها برای حل این مسئله طراحی شده‌اند:
- هر روتر فقط همسایه‌های مستقیم خود را می‌شناسد.
 - با تبادل پیام‌های کنترلی، اطلاعات کافی درباره بقیه شبکه به‌دست می‌آورد.
 - و از این اطلاعات برای ساخت Routing Table استفاده می‌کند تا در لحظه ارسال بتواند با یک نگاه به جدول، Next Hop مناسب را انتخاب کند.

۱-۵-۲ Link-State در برابر Distance-Vector

- در **Link-State** هر روتر تلاش می‌کند نقشه کامل شبکه را داشته باشد و سپس خودش با اجرای Dijkstra مسیرها را حساب می‌کند.
- در **Distance-Vector** هر روتر فقط می‌داند از دید خودش هزینه رسیدن به هر مقصد چقدر است و این اطلاعات را به‌صورت خلاصه (بردار فاصله) با همسایه‌ها ردوبدل می‌کند.

به همین دلیل:

• در Link-State:

- پیام‌ها مفصل‌تر هستند (حاوی اطلاعات کل همسایه‌ها و هزینه لینک‌ها)، اما تصمیم‌گیری‌ها بر پایه تصویر کامل‌تری از شبکه انجام می‌شود.
- همگرایی معمولاً سریع‌تر و مسیرها معمولاً بهینه‌اند، اما نگه داشتن و به‌روزرسانی نقشه کامل شبکه برای هر روتر هزینه بیشتری دارد.

• در Distance-Vector:

- پیام‌ها ساده‌تر و محلی‌تر هستند، اما به دلیل همین دید ناقص ممکن است مشکل‌هایی مثل Count-to-Infinity و همگرایی کند به وجود بیاید.
- برای شبکه‌های کوچک و ساده، پیاده‌سازی آن راحت‌تر است، اما برای شبکه‌های پیچیده‌تر مشاهده و دیباگ رفتار آن سخت‌تر می‌شود.

در این فاز، پیاده‌سازی **Distance-Vector کاملاً اختیاری و امتیازی** است، اما همین مقایسه مفهومی کمک می‌کند موقع طراحی شبیه‌ساز بهتر بفهمید چرا در قسمت اجباری روی Link-State تمرکز کرده‌ایم.

۱-۵-۳ OSPF و BGP

در دنیای واقعی:

• داخل یک سازمان (مثلاً یک دانشگاه یا شرکت بزرگ)، معمولاً از پروتکل‌هایی مثل OSPF استفاده می‌شود که خانواده Link-State هستند و روی بهینه بودن مسیرها تمرکز دارند.

• بین سازمان‌های مختلف در سطح اینترنت، پروتکل BGP استفاده می‌شود که بیش از آن‌که به کوتاه‌ترین مسیر فکر کند، بر پایه سیاست و قراردادهای تجاری مسیرها را انتخاب می‌کند.

در این پروژه **الزامی نیست** که OSPF یا BGP را دقیق شبیه‌سازی کنید، اگر علاقه داشته باشید، می‌توانید ایده‌های ساده‌ای از آن‌ها را در قالب بخش امتیازی به شبیه‌ساز خود اضافه کنید (مثلاً تعریف دو Area کوچک یا چند AS با سیاست‌های ساده) و ببینید چگونه همین شبیه‌سازی ساده به شما کمک می‌کند مفاهیم درون دامنه‌ای و بین دامنه‌ای را بهتر لمس کنید.

۱-۶ نقشه راه گام‌به‌گام و معماری کلی سیستم

این بخش یک پیشنهاد **غیرالزامی** برای ترتیب کار است تا فاز برای شما قابل مدیریت‌تر شود. می‌توانید دقیقاً همین مسیر را بروید یا با سلیقه خودتان کمی آن را تغییر دهید، اما توصیه می‌شود کلیت این ترتیب را حفظ کنید.

۱-۶-۱ گام ۰ - مطالعه مسئله و طراحی روی کاغذ

قبل از هر کدنویسی:

- یک بار این سند را کامل بخوانید و فصل مسیریابی کتاب Kurose & Ross (یا منبع اصلی درس) را مرور کنید.
- یک توپولوژی کوچک روی کاغذ بکشید (۵ تا ۸ روتر، چند لینک وزن‌دار، حداقل یک حلقه) و روی آن چند مسیر ممکن بین دو روتر را خودتان حساب کنید.
- مشخص کنید هر روتر چه اطلاعاتی باید در خود نگه دارد (همسایه‌ها، جدول مسیریابی، پایگاه Link-State، صف پیام‌ها و ...).
- تصمیم بگیرید مدل زمان شبیه‌ساز شما چگونه باشد (مثلاً یک حلقه ساده که در آن در هر دور، همه پیام‌های موجود پردازش می‌شوند و اگر پیامی جدید تولید شد، در دور بعدی پردازش می‌شود).

۱-۶-۲ گام ۱ - اسکلت شبیه‌ساز و مدل توپولوژی

در این گام هدف این است که بدون فکر کردن به LSA و Dijkstra، فقط بتوانید یک گراف ساده از روترها و لینک‌ها را بسازید.

- کلاس یا ساختار Router را تعریف کنید (شناسه، فهرست همسایه‌ها، جدول مسیریابی و ...). شناسه می‌تواند یک عدد صحیح یا یک رشته مانند "R1"، "R2" و غیره باشد.
- یک ساختار داده برای گراف بسازید (مثلاً Adjacency List که برای هر روتر، لیست همسایه‌ها و هزینه لینک به آن‌ها را نگه دارد).
- یک ماژول ساده برای تعریف توپولوژی بنویسید:

- در نسخه ساده، می‌توانید توپولوژی را به صورت ثابت در کد بنویسید.

- اگر خواستید، بعداً آن را به خواندن از فایل (مثلاً topology.txt) گسترش دهید.

- یک نقطه ورود (main) طراحی کنید که:

- روترها را بسازد.
- لینک‌ها را متصل کند.
- و در نهایت یک تابع `run_simulation()` را صدا بزند.

۱-۶-۳ گام ۲ - پیام‌های LSA و پایگاه Link-State

در این گام به شبیه‌ساز خود چشم‌انداز توپولوژی می‌دهید.

• قالب ساده LSA را طراحی کنید. مثلاً یک ساختار شامل:

- `origin_id`: شناسه روتر تولیدکننده.

- `seq`: شماره توالی LSA.

- `neighbors`: لیست (`neighbor_id, cost`).

• برای هر روتر، یک ساختار Link-State Database در نظر بگیرید که آخرین LSA معتبر هر روتر را نگه دارد (مثلاً یک map از `origin_id` به خود LSA).

• منطق انتشار LSA از هر روتر به همسایه‌هایش را در شبیه‌ساز پیاده‌سازی کنید:

- در شروع شبیه‌سازی، هر روتر یک LSA اولیه تولید و برای همه همسایه‌هایش ارسال می‌کند.

- وقتی روتر X یک LSA جدید از Y می‌گیرد، اگر `seq` آن جدیدتر بود، آن را در پایگاه خود ذخیره کرده و برای همسایه‌های دیگرش (به جز فرستنده) ارسال می‌کند.

- اگر LSA قدیمی یا تکراری باشد، آن را نادیده می‌گیرد.

• در پایان چند دور انتشار، یک تابع `debug_lsdb()` (مثلاً بنویسید که پایگاه Link-State چند روتر را چاپ کند تا مطمئن شوید همه تقریباً اطلاعات یکسانی دارند).

۱-۶-۴ گام ۳ - اجرای Dijkstra و ساخت Routing Table

در این گام، Link-State Database را به یک Routing Table واقعی تبدیل می‌کنید.

• تابعی بنویسید که روی گراف حاصل از Link-State Database الگوریتم Dijkstra را اجرا کند.

این تابع برای یک مبدأ مشخص (روتر جاری) اجرا می‌شود.

• خروجی Dijkstra معمولاً حداقل شامل دو ساختار است:

– $\text{dist}[v]$: هزینه کمینه تا هر گره v .

– $\text{parent}[v]$: گره قبلی در کوتاه‌ترین مسیر تا v .

- با استفاده از $\text{parent}[v]$ مسیر مبدأ تا هر مقصد را بازسازی کنید (با حرکت از مقصد به عقب تا مبدأ) و از اولین گره بعد از مبدأ به عنوان Next Hop استفاده کنید.
- جدول مسیریابی هر روتر را پر کرده و امکان چاپ آن را فراهم کنید. بهتر است دست‌کم یک تابع $\text{print_routing_table}(\text{router_id})$ داشته باشید که برای ارائه پروژه نیز به درد بخورد.
- برای اطمینان از صحت پیاده‌سازی Dijkstra، می‌توانید ابتدا آن را روی یک گراف بسیار کوچک (مثلاً ۳ یا ۴ گره) که جواب دستی آن را می‌دانید امتحان کنید.

۱-۶-۵ گام ۴ - ارسال یک بسته داده آزمایشی

در این گام نشان می‌دهید که Routing Table فقط برای زیبایی چاپ نشده و واقعاً می‌توان از آن برای هدایت یک بسته در شبیه‌ساز استفاده کرد.

- قالب ساده‌ای برای یک بسته داده تعریف کنید (فقط مبدأ و مقصد کافی است، در صورت تمایل می‌توانید payload نمادین هم اضافه کنید).
- تابعی بنویسید که بسته را از مبدأ برداشته و:

– Next Hop مقصد را از جدول مسیریابی روتر مبدأ بخواند.

– بسته را به روتر بعدی منتقل کند (با صدا زدن تابعی در شبیه‌ساز).

– همین کار را در روترهای میانی تکرار کند تا بسته به مقصد برسد یا مشخص شود مسیری وجود ندارد.

- مسیر طی‌شده (مثلاً $R1 \ R3 \ R5$) را روی خروجی چاپ کنید تا بتوانید شبیه‌ساز را به صورت ملموس امتحان کنید.

• تصمیم بگیرید اگر مقصد در Routing Table موجود نبود چه رفتاری انجام شود (مثلاً چاپ پیغام مقصد غیرقابل دسترس و Drop بسته).

۷-۱ بخش‌های امتیازی

۱-۷-۱ الگوریتم Distance-Vector شبیه RIP

- پیاده‌سازی یک پروتکل ساده Distance-Vector که در آن هر روتر بردار هزینه خود را دوره‌ای برای همسایه‌ها می‌فرستد.
- به‌روزرسانی جدول مسیریابی بر اساس Distance Vector های جدید.
- طراحی یک توپولوژی کوچک که در آن بتوانید رفتار Count-to-Infinity و راه‌حلهایی مثل Split Horizon را نشان دهید (در حد ساده).

۲-۷-۱ ویژگی‌های شبیه به OSPF

- تقسیم شبکه شبیه‌سازی‌شده به دو یا سه Area کوچک.
- محدود کردن انتشار کامل LSA ها به داخل هر Area.
- تعریف روترهای مرزی (ABR) که خلاصه اطلاعات هر Area را به Area های دیگر منتقل می‌کنند.

۳-۷-۱ شبیه‌سازی مسیریابی BGP میان چند AS

- مدل‌سازی چند AS مستقل، که هر کدام یک شبکه داخلی ساده و مجزا دارند.
- تعریف روترهای مرزی (Border Router) و تبادل پیام‌های BGP شامل AS-Path،
- پیاده‌سازی چند Policy ساده، برای نمونه:
 - اولویت دادن به کوتاه‌ترین AS-Path،
 - جلوگیری از عبور ترافیک از یک AS مشخص.

۴-۷-۱ بصری‌سازی توپولوژی و مسیرها

- طراحی رابط گرافیک کاربری برای نمایش الگوریتم‌ای مسیریابی به طور بصری.

۵-۷-۱ شبیه سازی خطاها

- شبیه‌سازی رخدادهایی مانند Link Failure، Link Recovery یا تغییر Cost یک لینک در زمان شبیه‌سازی.

- بازتولید LSA و به‌روزرسانی دوباره جدول‌های مسیریابی بعد از رخداد.
- جمع‌آوری آمار ساده مانند تعداد پیام‌های LSA یا تعداد بارهای اجرای Dijkstra.

۶-۷-۱ Equal-Cost Multi-Path و چند سناریوی آماده

- پشتیبانی از چند مسیر با هزینه برابر (ECMP) و تقسیم ساده بار بین آن‌ها.
- تعریف چند Scenario آماده (مثلا شبکه حلقه‌ای کوچک، شبکه مش کوچک، شبکه دو Area) و امکان انتخاب آن‌ها از طریق آرگومان برنامه یا فایل تنظیمات.

۸-۱ نکات تکمیلی

- سعی کنید کدی بنویسید که منطق پروتکل مسیریابی را از هسته شبیه‌ساز جدا کند، این کار توسعه بخش‌های امتیازی را ساده‌تر می‌کند.
- زبان برنامه‌نویسی کاملاً به انتخاب شماست، تنها کافی است ابزارهای حداقلی برای ساختار داده گراف و اجرای الگوریتم‌ها را داشته باشید.
- پیشنهاد می‌شود هر گام (تعریف گراف، LSA، Dijkstra، ارسال بسته) را جداگانه تست کنید و بعد سراغ گام بعدی بروید، از همان ابتدا همه‌چیز را یکجا ننویسید.
- برای دریافت کامل نمره حتماً باید ریپازیتوری گروهتون را در GitHub classroom از لینک اعلام شده در گروه بسازید و کامیت‌های مرتبی داشته باشید.
- هدف اصلی این فاز، درک عمیق شما از نحوه کار یک پروتکل مسیریابی لایه شبکه (به‌ویژه الگوریتم Link-State و Dijkstra) است، بنابراین استفاده از هرگونه منبع آموزشی (کتاب، اسلاید، مراجع آنلاین، ChatGPT و ...) برای یادگیری و ایده گرفتن مجاز است، اما مسئولیت درک، پیاده‌سازی صحیح و توانایی توضیح طرح نهایی کاملاً بر عهده خود شماست.
- هر گروه موظف است یک گزارش بسیار کوتاه و حداکثر یک صفحه A4 تهیه کرده و آن را با نامی مانند report.pdf در ریپازیتوری خود قرار دهد. این گزارش باید بیشتر روی چرا و تصمیم‌های طراحی شما تمرکز کند، نه روی جزئیات نحوی کد. پیشنهاد می‌شود گزارش شامل بخش‌های زیر باشد:

- یک پاراگراف کوتاه درباره هدف شما در این فاز و این‌که می‌خواستید به چه چیزی برسید.

- یک شرح گام‌به‌گام (خلاصه) از کارهایی که انجام داده‌اید، به ترتیب منطقی و تکاملی، مثلاً: ابتدا اسکلت شبیه‌ساز و مدل گراف را نوشتیم، بعد کلاس Router و لینک‌ها را اضافه کردیم، سپس پیام‌های LSA و منطق Flooding را پیاده‌سازی کردیم، بعد الگوریتم Dijkstra و ساخت Routing Table را اضافه کردیم، و در نهایت ارسال بسته داده آزمایشی و سناریوهای تست را پیاده‌سازی کردیم.

- چند زیربخش کوتاه درباره انتخاب‌های مهم طراحی: این‌که چرا این ساختار داده را برای گراف انتخاب کردیم، چرا LSA را این‌گونه مدل کردیم، چرا منطق Flooding یا اجرای Dijkstra را در ماژول جدا قرار دادیم و موارد مشابه.

- معرفی چند تابع یا ماژول کلیدی (نه همه توابع) با نگاه طراحی، مثلاً توضیح دهید تابعی که مسئول اجرای Dijkstra است چه ورودی و خروجی‌ای دارد و چگونه از آن برای ساخت Routing Table استفاده می‌کنید، یا ماژولی که وظیفه مدیریت صف پیام‌ها و انتشار LSA را دارد چه نقشی در شبیه‌ساز بازی می‌کند، تاکید بر منطق و تقسیم مسئولیت‌ها باشد، نه توضیح خط‌به‌خط کد.

• در جلسه ارائه، انتظار می‌رود بتوانید با تکیه بر همین گزارش کوتاه و درک مفهومی خود، شبیه‌ساز را به‌خوبی ارائه و توضیح دهید. نمره اصلی این فاز روی توضیح ایده‌ها، طراحی، منطق پروتکل شبیه‌سازی‌شده و توانایی پاسخ به سوالات مفهومی است.

۹-۱ سوالات پیشنهادی برای ارائه

این پرسش‌ها می‌توانند هم برای آمادگی ارائه پروژه و هم برای چک کردن درک خودتان از این فاز مفید باشند.

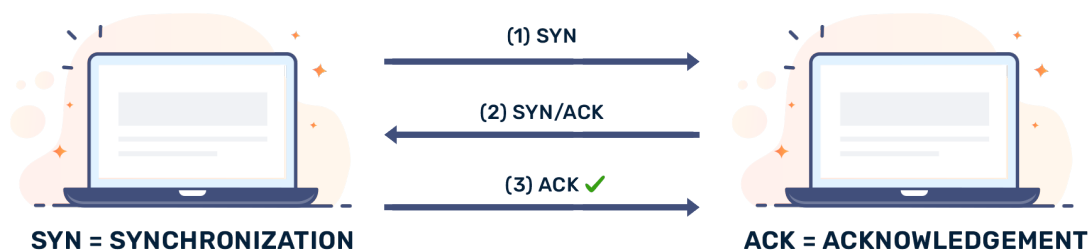
- در شبیه‌ساز شما، هر Router چه وضعیت‌هایی را نگه می‌دارد؟ (مثلا همسایه‌ها، جدول مسیریابی، پایگاه Link-State، صف پیام‌ها و ...) اگر بخواهید یک روتر جدید به توپولوژی اضافه کنید، چه چیزهایی را باید مقداردهی اولیه کنید؟
- منظور از این پروژه کاملاً شبیه‌سازی است در طراحی شما چیست؟ چه چیزهایی واقعی نیستند و فقط در منطق برنامه مدل شده‌اند؟ در ارائه چطور توضیح می‌دهید که شبیه‌ساز شما به IP Stack سیستم عامل وابسته نیست؟
- قالب پیام LSA در پیاده‌سازی شما چه فیلدهایی دارد و هر فیلد چه نقشی بازی می‌کند؟ اگر بخواهید یک فیلد جدید (مثلاً زمان تولید) اضافه کنید، چه استفاده‌ای از آن می‌کنید؟
- چگونه از Flooding بی‌پایان LSAها جلوگیری کرده‌اید؟ اگر یک LSA قدیمی دوباره به دست روتر برسد، دقیقاً چه شرط‌هایی را چک می‌کنید و چه رفتاری انجام می‌شود (ذخیره، نادیده گرفتن، یا فقط لاگ کردن)؟
- Link-State Database در کد شما چگونه سازمان‌دهی شده است؟ برای هر روتر تولیدکننده LSA چه اطلاعاتی نگه می‌دارید و اگر بخواهید گراف مورد نیاز Dijkstra را بسازید، این گراف را از روی LSDB چگونه استخراج می‌کنید؟
- الگوریتم Dijkstra را در پیاده‌سازی خودتان به‌طور کلی توضیح دهید: ورودی آن چیست (لیست گره‌ها، لیست یال‌ها، وزن‌ها) و خروجی آن را چگونه به Routing Table تبدیل می‌کنید؟ اگر یک مثال کوچک روی کاغذ داشته باشید، چگونه مرحله به مرحله وضعیت dist و parent را نشان می‌دهید؟
- اگر توپولوژی شبکه (مثلاً مجموعه لینک‌ها و هزینه‌ها) را در فایل/کد تغییر دهید و شبیه‌ساز را دوباره از ابتدا اجرا کنید، به ترتیب چه مراحل (ساخت گراف، انتشار LSA، به‌روزرسانی پایگاه Link-State، اجرای Dijkstra، ساخت Routing Table) در سیستم شما طی می‌شود تا به وضعیت پایدار جدید برسید؟
- در شبیه‌ساز شما، ارسال یک بسته داده از مبدأ تا مقصد قدم به قدم چگونه انجام می‌شود؟ در هر روتر چه تابعی صدا زده می‌شود، Next Hop چگونه از جدول مسیریابی خوانده می‌شود و چه اطلاعاتی (مثلاً شناسه روتر جاری) همراه بسته جابه‌جا می‌شود؟

- اگر بسته‌ای به روتر میانی برسد که در جدول مسیریابی آن روتر هیچ ورودی برای مقصد وجود ندارد، برنامه شما چه می‌کند و چرا چنین تصمیمی گرفته‌اید؟ آیا رفتار شما با چاپ پیام خطا و Drop بسته، یا با تلاش برای انتخاب یک مسیر پیش‌فرض (Default Route) متفاوت است؟
- چگونه بررسی می‌کنید که جدول‌های مسیریابی با توپولوژی تعریف‌شده سازگار هستند؟ برای یک مقصد مشخص، اگر هزینه مسیر بر اساس Routing Table را حساب کنید، چگونه آن را با هزینه کوتاه‌ترین مسیر روی گراف (که روی کاغذ یا با یک تابع تست جداگانه حساب شده) مقایسه می‌کنید؟
- در طراحی‌تان چه Invariant‌های ساده‌ای برای صحت سیستم در ذهن داشته‌اید (مثلاً هیچ مقصدی نباید Next Hop خودش باشد، یا هزینه هر ورودی جدول مسیریابی باید غیرمنفی باشد) و هنگام دیباگ چگونه این قاعده‌ها را بررسی کرده‌اید؟

سوالات بخش امتیازی (در صورت پیاده‌سازی)

- اگر الگوریتم Distance-Vector را هم پیاده‌سازی کرده‌اید، در یک توپولوژی نمونه چه تفاوت‌هایی بین رفتار آن و Link-State مشاهده کرده‌اید؟ از نظر سرعت همگرایی، تعداد پیام‌ها و سادگی دیباگ چه تفاوت‌هایی دیده‌اید؟
- اگر مفاهیم OSPF-like مانند Area را اضافه کرده‌اید، شبکه را به چه نواحی تقسیم کرده‌اید، ABRها چه نقشی در شبیه‌ساز شما دارند و چه تغییری در تعداد پیام‌ها/حجم اطلاعات نسبت به حالت بدون Area دیده‌اید؟
- اگر شبیه‌سازی ساده‌ای از BGP-like دارید، یک مثال از Policy که پیاده‌سازی کرده‌اید بنزید و توضیح دهید این Policy چگونه روی انتخاب مسیر اثر گذاشته است (مثلاً جلوگیری از عبور از یک AS مشخص یا ترجیح مسیر کوتاه‌تر از نظر تعداد AS).
- اگر بصری‌سازی توپولوژی را انجام داده‌اید، در زمان تغییر توپولوژی (مثلاً Link Failure در سناریوی امتیازی) این بصری‌سازی چگونه به شما در فهم رفتار پروتکل کمک می‌کند؟ آیا توانسته‌اید مسیر جدید را به صورت گرافیکی هایلایت کنید؟

فاز سوم: پیاده‌سازی پروتکل انتقال مطمئن (RDT) بر بستر UDP



۱-۲ شرح کلی فاز سوم

در این فاز، شما یک لایه انتقال مطمئن (Reliable Data Transfer) روی پروتکل UDP پیاده‌سازی می‌کنید. علاوه بر این، یک کانال شبیه‌سازی خطا برای مدل‌کردن Loss، Corruption و Delay طراحی می‌کنید و در نهایت یک Client و Server می‌سازید که از RDT شما همانند یک سوکت مطمئن استفاده کنند. به این ترتیب، خروجی این فاز یک ارتباط کامل کلاینت-سرور خواهد بود که روی UDP اجرا می‌شود، اما از دید لایه کاربردی مانند یک سوکت مطمئن رفتار می‌کند که می‌توانید از آن برای انتقال پیام استفاده کنید.

در کنار بخش اصلی، برای امکاناتی مانند پیاده‌سازی ساده‌ای از Congestion Control، اجرای Client و Server روی دو ماشین جداگانه در یک LAN، پشتیبانی از سطوح‌های مختلف اطمینان (Configurable Reliability Levels)، مدیریت چند اتصال هم‌زمان (Multi-Connection Support)، افزودن Handshake و Teardown تمیز برای مدیریت چرخه عمر اتصال، و پیاده‌سازی مدل‌های پیشرفته‌تر Rate Control امتیاز اضافی در نظر گرفته می‌شود.

۲-۲ اهداف پروژه

هدف اصلی این فاز درک عمیق و پیاده‌سازی عملی انتقال مطمئن داده‌ها (RDT) بر بستر UDP است، به‌گونه‌ای که با تبدیل تئوری ماشین‌های حالت به کد، پروتکل RDT 3.0 را توسعه دهید. در این مسیر باید با ایجاد یک معماری لایه‌ای کلاینت-سرور و شبیه‌سازی خطاهای شبکه (نظیر تأخیر، گم‌شدن و خرابی بسته‌ها)، یاد بگیرید چگونه لایه RDT با مدیریت خطا، باز ارسال و کنترل جریان، رابطی مطمئن و انتزاعی را در اختیار لایه کاربردی قرار می‌دهد.

۲-۳ تعاریف مورد نیاز

۲-۳-۱ مفاهیم پایه لایه انتقال

- **UDP:** پروتکلی در لایه انتقال که بدون برقراری اتصال (Connectionless) کار می‌کند و برای تحویل، ترتیب یا کنترل جریان هیچ تضمینی نمی‌دهد و داده را به صورت Datagram ارسال می‌کند.
- **Port:** عددی که به همراه IP، یک برنامه مشخص روی یک میزبان را معرفی می‌کند و کمک می‌کند چند برنامه مختلف روی یک ماشین از طریق شبکه قابل دسترس باشند.
- **Datagram:** یک پیام مستقل UDP است که به عنوان یک واحد کامل ارسال و دریافت می‌شود، یعنی یا به مقصد می‌رسد یا در مسیر گم می‌شود و شکسته یا تکه‌تکه نمی‌شود.
- **Payload:** بخش داده‌ای داخل Datagram که بالاتر از هدرهای پروتکل‌ها قرار می‌گیرد و برای ما (به عنوان طراح RDT) محتوای اصلی پیام را تشکیل می‌دهد.

۲-۳-۲ مفاهیم RDT

- **RDT 3.0:** نسخه‌ای از پروتکل انتقال مطمئن بر پایه Stop-and-Wait که با تکیه بر Sequence Number، پیام‌های ACK و مکانیزم Timeout می‌تواند با Loss و Corruption مقابله کند.
- **Stop-and-Wait:** الگویی که در آن فرستنده در هر لحظه فقط یک بسته در حال انتقال دارد و بعد از ارسال آن، تا دریافت ACK منتظر می‌ماند و سراغ بسته بعدی نمی‌رود.
- **Action, Event, State Machine:** مدلی برای توصیف رفتار پروتکل که در آن در هر حالت، رخدادهایی مانند rdt_rcv, rdt_send یا Timeout باعث اجرای کنش‌هایی مانند deliver_data یا udt_send, make_pkt می‌شوند.
- **Sequence Number:** عدد کوچکی که به هر بسته نسبت داده می‌شود تا گیرنده بتواند تشخیص دهد بسته جدید است، تکراری است یا خارج از ترتیب مورد انتظار دریافت شده است.
- **ACK و NAK:** پیام‌های بازخورد گیرنده به فرستنده، ACK تأیید می‌کند بسته به درستی دریافت شده و NAK یا رفتار معادل آن اعلام می‌کند که گیرنده بسته را خراب دریافت کرده یا قادر به استفاده از آن نیست.

- **Checksum:** مقداری که روی هدر و داده بسته محاسبه می‌شود و گیرنده با محاسبه دوباره آن می‌تواند Corruption را تشخیص دهد.

۲-۳-۳ مفاهیم مدل خطا

- **Loss:** حالتی که در آن یک بسته در میانه راه گم می‌شود و هرگز به گیرنده نمی‌رسد (برای داده و برای پیام‌های کنترلی مثل ACK ممکن است رخ دهد).
- **Corruption:** وضعیتی که در آن در مسیر انتقال، برخی بیت‌ها یا بایت‌های بسته تغییر می‌کنند و Checksum دریافتی با داده موجود ناسازگار می‌شود.
- **Delay:** حالتی که بسته با تأخیر بیش از حد معمول به گیرنده می‌رسد و ممکن است باعث انقضای Timeout در سمت فرستنده شود.
- **Reordering:** حالتی که بسته‌ها به ترتیبی متفاوت از ترتیب ارسال، به گیرنده تحویل داده می‌شوند و پروتکل باید بتواند با وجود این جابجایی، داده را به صورت مرتب تحویل لایه بالا بدهد.

۲-۴-۲ مرور مفاهیم

۲-۴-۲ چرا روی UDP به RDT نیاز داریم؟

پروتکل UDP فقط یک Checksum ابتدایی برای تشخیص خرابی بیت‌ها دارد، اما هیچ مکانیزمی برای دوباره ارسال کردن بسته‌های گمشده، تضمین ترتیب رسیدن داده‌ها، کنترل سرعت یا جلوگیری از ازدحام ندارد. بنابراین، اگر بخواهیم فایل، پیام یا به صورت کلی داده را به صورت قابل اعتماد روی UDP منتقل کنیم، باید لایه‌ای اضافه کنیم که خرابی و گم‌شدن بسته‌ها را تشخیص و جبران کند، از تکرار جلوگیری کند، ترتیب را حفظ کند و سرعت ارسال را تنظیم کند. این لایه همان RDT است که با قرار گرفتن روی UDP، آن را به یک سرویس قابل اتکا تبدیل می‌کند.

۲-۴-۲ مروری بر کتاب Kurose & Ross

در کتاب Kurose & Ross پروتکل RDT به صورت تدریجی از یک مدل ایده‌آل به مدل‌های واقع‌گرایانه‌تر گسترش می‌یابد. در rdt1.0 فرض می‌شود کانال نه بسته را خراب می‌کند و نه چیزی را گم می‌کند، بنابراین فرستنده فقط داده را بسته‌بندی و ارسال می‌کند و گیرنده نیز آن را مستقیماً به لایه بالا تحویل می‌دهد. سپس در rdt2 خرابی بیت‌ها مطرح می‌شود و برای تشخیص آن Checksum اضافه می‌شود و گیرنده در پاسخ هر بسته ACK یا NAK می‌فرستد و فرستنده در صورت دریافت NAK بسته را دوباره ارسال می‌کند (با اضافه شدن شماره توالی صفر و یک نیز امکان تشخیص Duplicate فراهم می‌شود).

اما نقطه اصلی تحول در rdt3.0 است که علاوه بر خرابی، مساله گم شدن بسته یا ACK را هم در نظر می‌گیرد و به همین دلیل فرستنده بعد از ارسال هر بسته یک Timer تنظیم می‌کند و اگر قبل از پایان زمان ACK معتبر نرسد آن بسته را دوباره می‌فرستد و کاری ندارد که مشکل ناشی از گم شدن داده بوده یا گم شدن ACK. گیرنده نیز با بررسی Checksum و شماره توالی فقط بسته‌های صحیح و جدید را تحویل می‌دهد و در باقی موارد آخرین ACK معتبر را تکرار می‌کند. در نتیجه تنها یک بسته در هر لحظه در جریان است و با استفاده از شماره توالی‌های صفر و یک این نسخه عملاً همان Alternating Bit Protocol و نمونه کلاسیک یک پروتکل Stop-and-Wait قابل اعتماد روی یک کانال نامطمئن است.

۲-۴-۳ کانال شبیه‌سازی خطا

برای آزمایش رفتار RDT بدون وابستگی به شرایط واقعی اینترنت می‌توانید منطق ایجاد خطا را مستقیماً در بخش کلاینت یا سرور و دقیقاً در همان تابعی قرار دهید که بسته‌ها روی UDP ارسال یا دریافت می‌شوند و هر کدام خواستید می‌تواند هنگام کار خود نقش شبکه بد را ایفا کند و خطا بسازد. انواع خطاهایی که باید شبیه‌سازی کنید شامل تغییر تصادفی بیت‌های بسته، دراپ کردن

بسته و ارسال تکراری آن هستند و در بخش امتیازی هم می‌توانید تاخیر تصادفی برای بسته‌ها بگذارید. توجه کنید که احتمال وقوع هر خطا باید با چند پارامتر قابل کنترل باشد.

۵-۲ بخش‌های الزامی

۱. پیاده‌سازی اولیه کلاینت و سرور روی UDP

اول یک Client و یک Server ساده بسازید که روی یک پورت UDP با هم کار کنند. در این مرحله کلاینت فقط می‌تواند یک پیام ساده برای سرور بفرستد و سرور آن را روی کنسول چاپ کند. پیاده‌سازی ارتباط دوطرفه الزامی نیست.

۲. اضافه کردن منطق خطا داخل کلاینت یا سرور

در گام بعدی باید یک شبکه بد ولی قابل کنترل بسازید. (نیازی به ساخت پروکسی جدا نیست، می‌توانید مستقیماً داخل همان جایی که بسته‌ها را روی UDP ارسال یا دریافت می‌کنید این کار را انجام دهید). همچنین می‌توانید به دلخواه، این منطق را در سمت کلاینت یا سرور (یا هر دو) و دقیقاً در تابعی که برای send یا recv استفاده می‌کنید اضافه کنید تا هر بسته قبل از ارسال یا هنگام دریافت، از این فیلتر خطا عبور کند.

انواع خطاهایی که باید شبیه‌سازی شوند:

- **تغییر بیت در Payload:** بخشی از محتوای بسته را به صورت تصادفی عوض کنید تا Corruption داشته باشید.
- **Drop شدن بسته:** بسته را اصلاً ارسال نکنید یا آن را نادیده بگیرید تا گم شدن شبیه‌سازی شود.
- **ارسال تکراری همان بسته:** گاهی همان بسته را دو بار (یا چند بار) پشت سر هم بفرستید تا Duplicate ایجاد شود.

پارامترهای پیشنهادی برای کنترل خطا:

- P_{drop} : احتمال Drop شدن هر Datagram.
- $P_{corrupt}$: احتمال Corrupt شدن Payload.
- $P_{duplicate}$: احتمال ارسال تکراری یک بسته.

بعد از این گام، اگر چند بار پیام ساده ارسال کنید باید بتوانید ببینید که بعضی پیام‌ها گم می‌شوند، بعضی خراب می‌رسند یا تکراری دریافت می‌شوند و متوجه شوید که انتقال خام UDP غیرقابل اعتماد است.

۳. پیاده‌سازی RDT 3.0 (Stop and Wait) بر بستر UDP

در این گام RDT 3.0 را مطابق FSM های کتاب Kurose & Ross برای یک اتصال ساده پیاده‌سازی می‌کنید. در پایان، باید روی همین کانال UDP که بسته‌ها را گم می‌کند یا خراب می‌کند، سرویسی داشته باشید که داده را درست، بدون تکرار و به ترتیب به مقصد تحویل دهد، حتی اگر مجبور شود چندین بار یک بسته را باز ارسال کند.

مفاهیم کلیدی طراحی RDT 3.0:

- شماره دوتایی Sequence Number (۰ و ۱) به صورت Alternating Bit.
- پیام‌های ACK (و در صورت تمایل NAK) برای بازخورد گیرنده.
- Checksum برای تشخیص خراب شدن بسته‌ها.
- Timeout (و در صورت تمایل تخمین RTT) برای تشخیص گم شدن بسته یا ACK و بازارسال بسته جاری.
- پیاده‌سازی دقیق State Machine فرستنده و گیرنده مطابق نمودارهای کتاب.

الف) API لایه RDT

یک API شبیه سوکت برای خودتان تعریف کنید تا لایه بالاتر فقط با یک رابط ساده کار کند؛ مثلاً توابعی برای ایجاد یک Endpoint، اتصال به آدرس و پورت، ارسال داده، دریافت داده و در نهایت بستن اتصال در نظر بگیرید تا جزئیات پیاده‌سازی RDT پشت این توابع پنهان بماند. سپس تمام منطق Timeout، NAK/ACK، Sequence Number و بازارسال پشت این API پنهان می‌شود و در پشت صحنه روی یک UDP Socket و عبور از همان منطق خطا اجرا می‌گردد.

ب) قالب سگمنت‌های RDT

برای هر سگمنت RDT یک قالب مشخص برای هدر و داده تعریف کنید، مثلاً:

- Type: نوع پیام (NAK، ACK، DATA).
- Seq: شماره توالی (۰ یا ۱).
- Length: طول داده.
- Checksum: جمع کنترلی روی هدر و داده.
- Payload: داده کاربردی.

نحوه سریال‌سازی (struct یا JSON یا آرایه bytes) به انتخاب شماست، اما تبدیل بین ساختار داخلی و باینری باید دقیق و بدون ابهام باشد.

ج) منطق فرستنده (Sender FSM)

• **حالت انتظار برای داده از لایه بالا:** روی `rdt_send(data)` بسته با Seq فعلی می‌سازید، روی UDP می‌فرستید، Timer را شروع می‌کنید و به حالت انتظار برای ACK می‌روید.

• حالت انتظار برای ACK:

- اگر سگمنت رسید و خراب بود یا ACK برای Seq دیگری بود آن را نادیده می‌گیرید.
- اگر ACK معتبر برای Seq فعلی بود Timer را متوقف می‌کنید، Seq را بین ۰ و ۱ عوض می‌کنید و به حالت انتظار برای داده برمی‌گردید.
- اگر Timeout شد همان `sndpkt` را دوباره می‌فرستید و Timer را از نو راه می‌اندازید.

د) منطق گیرنده (Receiver FSM)

گیرنده در حالت انتظار برای Seq = 0 یا انتظار برای Seq = 1 است:

• **اگر بسته خراب باشد:** می‌توانید ACK آخرین Seq درست را دوباره بفرستید یا NAK بدهید.

• اگر بسته سالم و Seq برابر Expected Seq باشد:

- داده را با `deliver_data` به لایه بالا تحویل می‌دهید،
- یک ACK برای همین Seq می‌فرستید،
- Expected Seq را بین ۰ و ۱ عوض می‌کنید.

• اگر بسته سالم اما Seq تکراری باشد:

- داده را دوباره تحویل نمی‌دهید،
- فقط همان ACK قبلی را دوباره ارسال می‌کنید تا فرستنده متوجه شود ACK قبلی به دستش نرسیده است.

۶-۲ بخش‌های امتیازی

- دو طرفه کردن ارتباط کلاینت و سرور (به صورت reliable).
- اضافه کردن تأخیر روی بسته: می‌توانید پارامترهایی مثل P_delay و $MaxDelay$ تعریف کنید و با احتمال مشخص ارسال بسته را تا حداکثر یک زمان تصادفی عقب بیندازید تا اثر شبکه کند یا شلوغ را ببینید.
- پیاده‌سازی Selective Repeat یا Go-Back-N.
- افزودن Congestion Control ساده مثلاً بر پایه الگوریتم AIMD.
- اجرای Client و Server روی دو ماشین واقعی در یک LAN.
- پشتیبانی از چند اتصال هم‌زمان (Multi-Connection Support) در سمت سرور.
- افزودن Handshake و Teardown ساده برای مدیریت برقراری و پایان اتصال (پیام‌های FIN/ACK/SYN و غیره).
- هرگونه قابلیت‌هایی که پروتکل RDT را به TCP نزدیک‌تر کند.
- استفاده از Raw Socket به جای UDP: بسته‌ها را به فرمت بسته‌های TCP به عنوان پیلود لایه ۳ ارسال کنید. در این حالت مجبور خواهید بود Handshake سه مرحله‌ای TCP را به طور کامل پیاده‌سازی کنید. (با توجه به محدودیت‌های ویندوز برای سوکت‌های Raw نیاز به سیستم‌عامل لینوکس خواهید داشت).

۷-۲ نکات تکمیلی

- برای دریافت کامل نمره حتما باید ریپازتوری گروهتون را در GitHub classroom از لینک اعلام شده در گروه بسازید و کامیت‌های مرتبی داشته باشید.
- استفاده از نرم‌افزار Wireshark (که در تمرین لایه اپلیکیشن با آن آشنا شدید) برای دیباگ این پروژه بسیار کمک‌کننده خواهد بود.
- هدف اصلی این پروژه درک شما از روش کار یک پروتکل انتقال مطمئن داده است، در نتیجه استفاده از هر گونه منبع برای انجام پروژه (از جمله چت‌بات‌ها) مجاز است.
- استفاده از کتابخانه‌های آماده (مثلا برای حساب کردن checksum) مجاز نمی‌باشد.
- هر گروه موظف است یک گزارش بسیار کوتاه و حداکثر یک صفحه A۴ تهیه کند و آن را در ریپازیتوری خود (مثلا با نام report.pdf) قرار دهد. این گزارش باید بیشتر روی چرایی ها و تصمیم های طراحی تمرکز کند، نه روی جزئیات نحوی کد. پیشنهاد می شود گزارش شامل بخش های زیر باشد:
 - یک پاراگراف کوتاه درباره این که هدف این فاز چه بوده و شما می خواستید به چه چیزی برسید.
 - یک شرح گام به گام از کارهایی که انجام داده اید (به ترتیب منطقی و تکاملی)، مثلا: اول یک UDP Client/Server ساده نوشتیم، بعد مدل خطا را اضافه کردیم، بعد RDT 3.0 را روی آن سوار کردیم، سپس ...
 - توضیح چرایی انتخاب ها: این که این کار را انجام دادیم چون ...، این ساختار سگمنت را انتخاب کردیم چون ...، این تابع یا مازول را نوشتیم تا مسئولیت X را جدا کنیم، و غیره.
 - معرفی چند تابع یا مازول کلیدی (نه همه توابع) با نگاه طراحی: مثلا تابع send_rdt را نوشتیم تا منطق Timeout و بازارسال پشت یک API ساده پنهان شود، نه توضیح خط به خط کد.
- در جلسه ارائه، انتظار می رود بتوانید از همین گزارش کوتاه و درک مفهومی خودتان دفاع کنید و نمره اصلی روی توضیح ایده ها، طراحی و منطق پروتکل شما است. توضیح دادن مستقیم از روی کد (خواندن خط به خط) به عنوان ارائه اصلی پذیرفته نمی شود.

۸-۲ سؤالات پیشنهادی برای ارائه

برای ارائه این فاز آمادگی پاسخ‌گویی به پرسش‌های زیر را داشته باشید:

- چرا به RDT نیاز داریم وقتی UDP خودش Checksum دارد؟
- تفاوت‌های اصلی RDT 3.0 و TCP چیست؟
- در پیاده‌سازی RDT شما، ساختار هر سگمنت چگونه است؟ چه فیلدهایی در هدر RDT دارید (نوع پیام، Seq، طول، Checksum و ...) و چرا؟
- وقتی یک بسته داده گم می‌شود، چه اتفاقی در فرستنده و گیرنده شما رخ می‌دهد؟ اگر ACK گم شود چه تفاوتی دارد؟

سؤالات بخش امتیازی

- در پیاده‌سازی Sliding Window (اگر پیاده‌سازی کرده‌اید):
 - چگونه Timerها را مدیریت می‌کنید (یک تایمر برای کل پنجره یا یک تایمر برای هر بسته)؟
 - بسته‌های خارج از ترتیب را در سمت گیرنده چگونه مدیریت و در چه زمانی تحویل لایه بالا می‌دهید؟
- اگر کنترل ازدحام ساده‌ای پیاده‌سازی کرده‌اید، شکل تغییرات اندازه پنجره یا نرخ ارسال در طول زمان چگونه است و رویداد Loss چه تأثیری روی آن دارد؟
- اگر از چند اتصال هم‌زمان پشتیبانی می‌کنید، چگونه بین حالت‌های مختلف RDT برای هر اتصال تفکیک قائل می‌شوید و داده‌های آن‌ها را جدا نگه می‌دارید؟
- اگر Handshake و Teardown ساده‌ای پیاده‌سازی کرده‌اید، توالی پیام‌های SYN، SYN-ACK، ACK و همچنین FIN-ACK/FIN چگونه است و در هر مرحله چه وضعیت‌هایی در فرستنده و گیرنده تغییر می‌کند؟